

AVL TREE

An AVL tree is a self-balancing binary search tree in Data Structures where the height difference between the left and right subtree of any node is at most 1. The difference is called Balance Factor.

AVL abbreviation is Adelson - Velsky and Landis, invented it in 1962.

Balance Factor:

For any node

$$BF = \text{Height of Left Subtree} - \text{Height of Right Subtree}$$

Allowed (Balanced) Values:

AVL tree is balanced if every node has balance factor:

-1, 0, or +1 only

- **-1** → Right subtree is slightly taller
- **0** → Both subtrees have equal height
- **+1** → Left subtree is slightly taller

Not Allowed(Unbalanced):

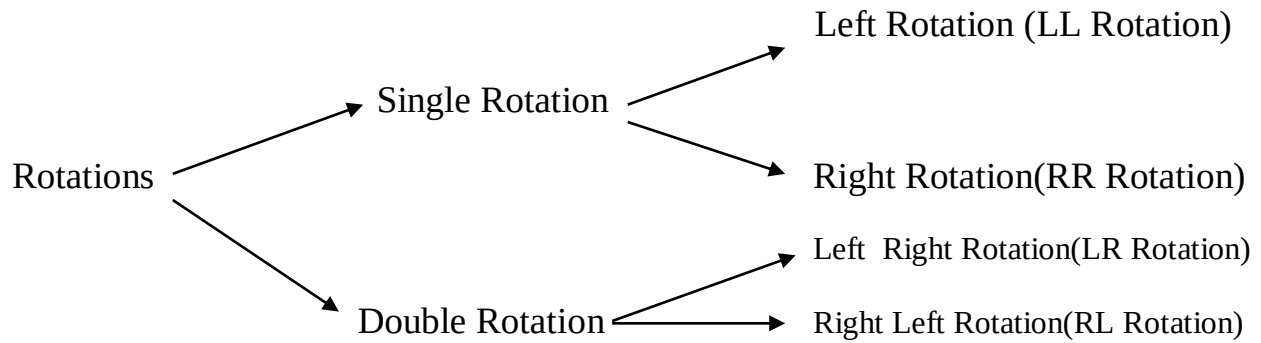
If a node gets these values -2,+2

Therefore balance factor goes outside this range, the tree performs **rotations** to fix itself.

TYPES OF ROTATIONS

- AVL rotation means rearranging nodes in an AVL tree to make it balanced.
- If a node gets these values -2 ,+2, then tree becomes **unbalanced**, and rotations are performed to fix it.

To maintain balance, AVL trees use four types of rotations:



a) Single Rotation:

In an AVL tree, a single rotation means fixing an imbalanced tree using one rotation only.

Or

Single rotation balances the tree using just one left or right move.

There are Two Types:

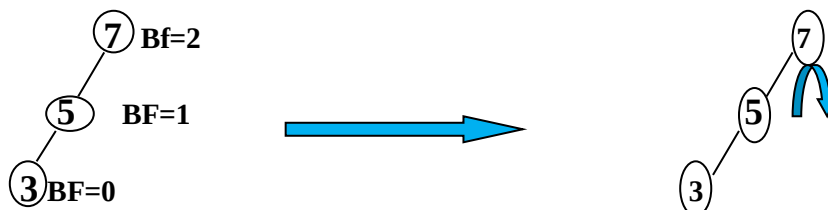
1. Left Left Case
2. Right Right Case

1. Left Left Case (Right Rotation):

- It happens when a node is inserted in the **left subtree of left child**,
- It happens when the tree is **left-heavy**.
- Fix: rotate **right**
- LL imbalance → Right rotation

Before Rotation:

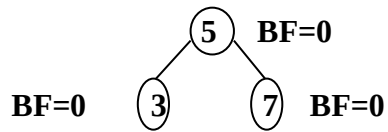
Example: insert 7,5,3



Tree is imbalanced

To make balanced we use **Right Rotation** which moves Nodes one position to right

After Right Rotation :



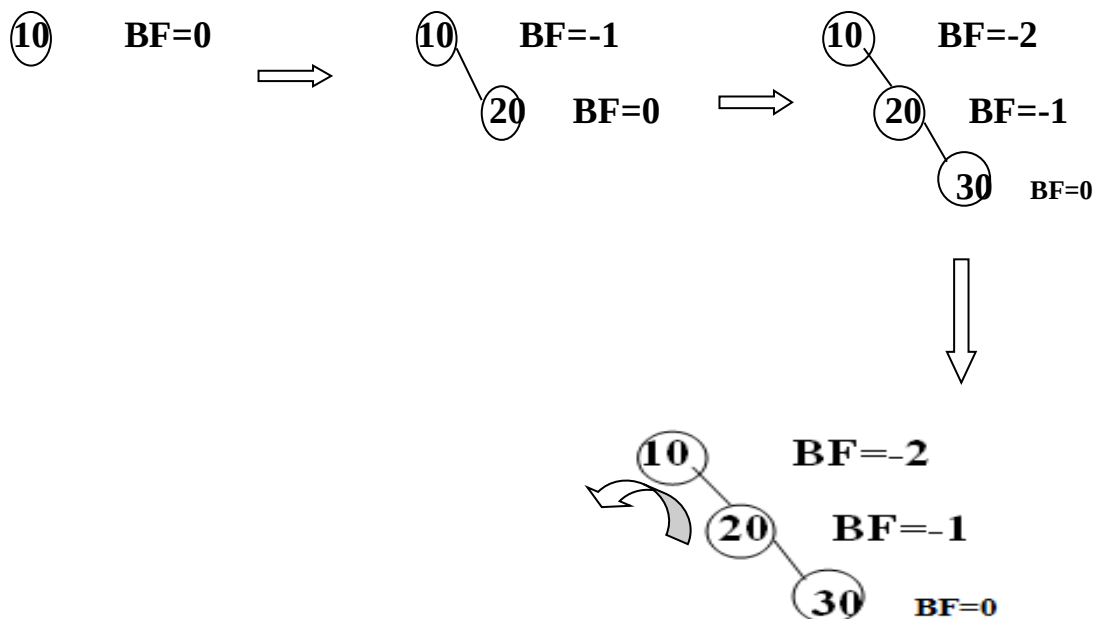
Tree is balanced

2. Right Right Case(Left Rotation):

- It happens when the tree is **right-heavy**
- It happens when a node is inserted in the right subtree of the right child.
- It happens when the tree is **right-heavy**.
- Fix: rotate **left**
- RR imbalance → Left rotation

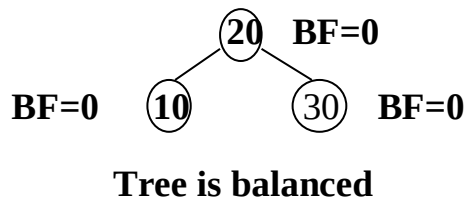
Example: Insert: 10 ,20 ,30

Before Rotation:



Tree is imbalanced and apply left Rotation

After Left Rotation:



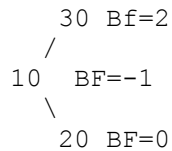
3. Left-Right Case(LR Rotation):

It happens when a node is inserted in the right subtree of the left child.

Example:

Insert: 30 → 10 → 20

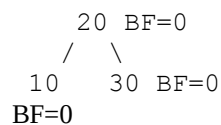
Before rotation:



Step 1: Left rotation on 10

Step 2: Right rotation on 30

After rotation:



4. RL Rotation (Right-Left Case)

It happens when a node is inserted in the **left subtree of the right child**.

Example:

Insert: 10 → 30 → 20

Before rotation:

```
10 BF=-2
  \
   30 BF=1
  /
 20 BF=0
```

Step 1: Right rotation on 30

Step 2: Left rotation on 10

After rotation:

```
      20 BF=0
     /  \
BF=0 10   30 BF=0
```

OPERATIONS OF AVL TREE

1. BALANCE FACTOR:

For every node:

Balance Factor = Height of Left Subtree – Height of Right Subtree

Allowed values:

-1
0
+1

If the node values are -2, +2, then tree becomes unbalanced and rotations are performed.

2. SEARCH OPERATION:

Searching in an AVL tree is similar to searching in a BST.

Steps

1. Compare the key with the root.
2. If equal → element found.
3. If smaller → search left subtree.
4. If larger → search right subtree.

Time Complexity

- **Best/Average/Worst:** $O(\log n)$

Example

Search 25:

```
      30
     /  \
    20   40
     \
     25
```

- $25 < 30 \rightarrow$ move left

- $25 > 20 \rightarrow$ move right
- Found 25

3.INSERTION OPERATION:

Insertion in an AVL tree is performed in two phases:

1. **Insert the node** using Binary Search Tree (BST) rules.
2. **Balance the tree** if any node becomes unbalanced.

Steps of AVL Insertion

Step 1: Insert as in BST

- If key is smaller $\rightarrow$ go to left subtree.
- If key is larger $\rightarrow$ go to right subtree.

Step 2: Update Height

Step 3: Check Balance Factor

For each node:

- If balance factor is -1 , 0 , or $+1 \rightarrow$ tree is balanced.
- Otherwise, rotation is required.

Rotation Cases in AVL Insertion

There are four Cases of Imbalance:

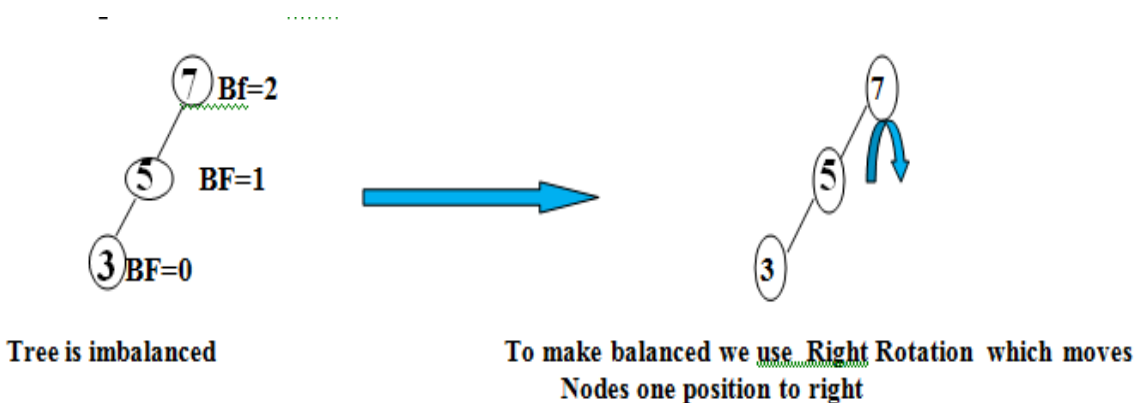
1. LL Case(Right Rotation)
2. RR Case(Left Rotation)
3. LR Case(Left Right Rotation)
4. RL Case(Right Left Rotation)

1. Left Left(LL) Case(Right Rotation):

- It happens when a node is inserted in the **left subtree of left child**,

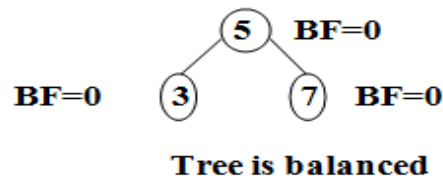
Before Rotation:

Insert 7,5,3



- Balance factor of 30 becomes $+2$. So Rotation should performed
- **Operation:** Perform Right Rotation on 7

After Rotation:



2. Right-Right (RR) Case(Left Rotation):

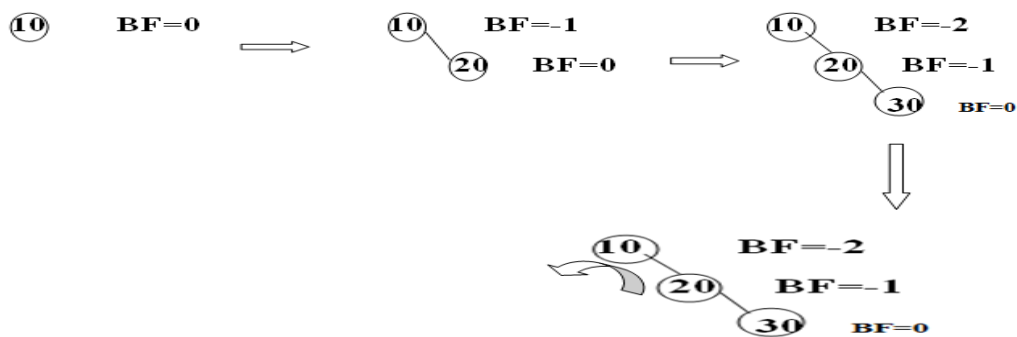
It happens when a node is inserted in the right subtree of the right child.

Example:

Insert: 10 ,20 ,30

Before Rotation:

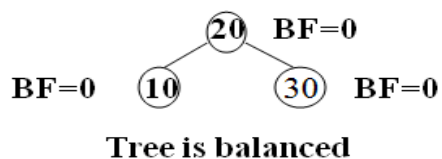
Before Rotation:



Tree is imbalanced and apply left Rotation

- Balance factor of 10 becomes -2.
- **Operation:** Perform Left Rotation on 10.

After Left Rotation:



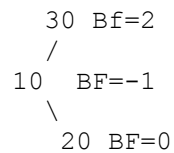
3. Left-Right (LR) Case(LR Rotation)

It happens when a node is inserted in the right subtree of the left child.

Example:

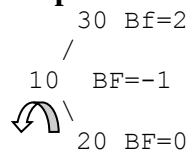
Insert: 30 ,10 ,20

Before rotation:

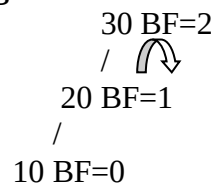


- Balance factor of 10 becomes +2.
- **Operations:**
 - 1.Left Rotation on 10
 - 2.Right Rotation on 30

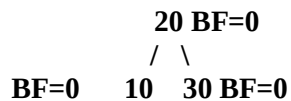
Step 1: Left Rotation on 10



Step2 Right Rotation on 30



After right rotation:



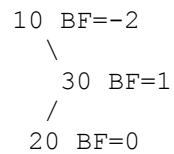
4. Right-Left (RL)Case(RL Rotation)

It happens when a node is inserted in the **left subtree of the right child**.

Example:

Insert: $10 \rightarrow 30 \rightarrow 20$

Before rotation:

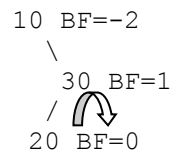


- Balance factor of 10 becomes -2.
- **Operations:**

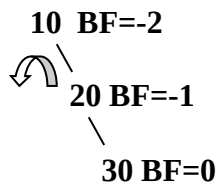
1. Right rotation on 30

Left rotation on 10

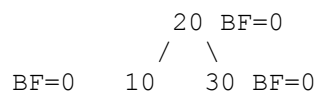
Step 1: Right rotation on 30



Step 2: Left rotation on 10



After Left rotation:



4.DELETION OPERATION:

Deletion in an AVL Tree is similar to deletion in a Binary Search Tree (BST), but after deleting a node, the tree must be checked for balance and rotations may be required.

Steps

1. Delete node as in BST.
2. Update heights.
3. Find the balance factor of each node.
4. Perform rotations if the tree becomes unbalanced.

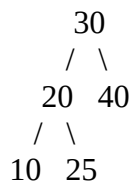
Cases in AVL Deletion:

After deletion, imbalance can occur in four ways:

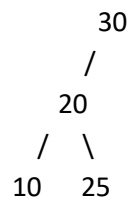
Imbalance Type	Rotation Required
LL Case	Right Rotation
RR Case	Left Rotation
LR Case	Left Rotation + Right Rotation
RL Case	Right Rotation + Left Rotation

Example of AVL Deletion:

Delete node :40



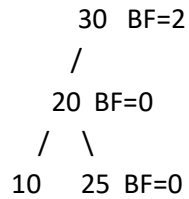
Step1: Delete a node in BST



Step 2: Check Balance Factor

For node 30:

- Height of left subtree = 2
- Height of right subtree = 0

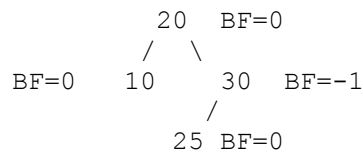


Tree is unbalanced

This is an LL Case because imbalance occurs in the left subtree of the left child.

Step 3: Perform Right Rotation

After right rotation at 30:



Now the AVL tree becomes balanced again.

5. TRAVERSAL OPERATIONS:

AVL trees support all standard tree traversals:

Traversal	Order
Preorder	Root → Left → Right
Inorder	Left → Root → Right
Postorder	Left → Right → Root
Level Order	Breadth-first

Applications of AVL Trees

1. Database indexing
2. Memory management systems
3. Symbol tables in compilers
4. Dictionary and searching applications

NOTE: Refer time complexity and space complexity in u r class work